

Responsive Design: Guía Teórica Completa

1. Qué es Responsive Design

El diseño responsive busca que una interfaz se adapte automáticamente a distintos tamaños de pantalla sin romperse ni obligar al usuario a hacer zoom como si fuera 2010. Su objetivo es mantener accesibilidad, legibilidad y funcionalidad en cualquier dispositivo.

2. Conceptos Fundamentales

Mobile-first vs Desktop-first

- **Mobile-first:** se parte del diseño para móviles y se escala hacia pantallas mayores.

```
@media (min-width: 768px) { }
```

- **Desktop-first:** se diseña grande y luego se ajusta a pantallas pequeñas.

```
@media (max-width: 768px) { }
```

Mobile-first hoy es el estándar.

Breakpoints

Son puntos de corte donde el layout cambia. Ejemplo típico: - 480px teléfonos pequeños - 768px tablets - 1024px notebooks - 1280px desktops grandes

No son reglas fijas; se ajustan al proyecto.

3. Media Queries

```
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

También existen media queries más específicas:

```
@media (orientation: landscape) { }  
@media (prefers-color-scheme: dark) { }  
@media (hover: none) { }
```

4. Unidades para Responsive

Porcentajes

```
width: 100%;
```

Viewport units

```
width: 100vw;  
height: 100vh;
```

Unidades fluidas (clamp, min, max)

```
font-size: clamp(1rem, 2vw, 2rem);
```

Permiten escalas reales sin perder control.

5. Tipografía Responsive

Métodos comunes

- Usar `clamp()` para textos que se adapten.
- Establecer line-height consistente.
- No fijar tipografías en px para móviles.

Ejemplo recomendado:

```
h1 {  
  font-size: clamp(2rem, 5vw, 4rem);  
}
```

6. Imágenes y Medios

Imágenes fluidas

```
img {  
  width: 100%;  
  height: auto;  
}
```

object-fit

```
img {  
  object-fit: cover;  
}
```

picture + source (según resolución)

```
<picture>  
  <source srcset="img-large.jpg" media="(min-width: 1024px)">  
    
</picture>
```

7. Layout Responsive

Flexbox con media queries

```
.container {  
  display: flex;  
  gap: 1rem;  
}  
  
@media (max-width: 768px) {  
  .container { flex-direction: column; }  
}
```

Grid con auto-fit

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
}
```

```
gap: 1rem;
}
```

Grid permite responsive sin media queries.

8. Navegaciones Responsivas

Menú hamburguesa básico

```
.nav {
  display: flex;
  justify-content: space-between;
}

.menu { display: flex; }

@media (max-width: 768px) {
  .menu { display: none; }
}
```

La lógica con JavaScript aparece en proyectos reales.

9. Containers y Wrappers

Usar un contenedor evita que el diseño se expanda demasiado.

```
.container {
  width: min(90%, 1200px);
  margin: 0 auto;
}
```

Esta técnica optimiza lectura y mantiene la armonía visual.

10. Diseño Fluido vs Adaptativo

- **Fluido:** se estira con el viewport.
 - **Adaptativo:** cambia en puntos específicos.
 - **Responsive moderno:** mezcla ambos.
-

11. Buenas Prácticas

- Probar constantemente en tamaños reales.
 - No crear 10 breakpoints sin razón.
 - Mantener tipografías con `clamp()`.
 - Evitar imágenes gigantes: usar formatos optimizados.
 - Crear layouts que no dependan del zoom.
 - Preferir mobile-first.
-

12. Checklist Rápido

- ☐ ¿Tu layout se rompe al bajar de 400px?
 - ☐ ¿Estás usando unidades fluidas?
 - ☐ ¿Las imágenes son responsive?
 - ☐ ¿El contenido tiene un max-width razonable?
 - ☐ ¿Los breakpoints están justificados por el diseño y no por superstición?
-

13. Para Practicar

- Landing page mobile-first
 - Grid de productos autoajustable
 - Navbar con menú colapsable
 - Hero section con tipografía fluida
 - Blog con layout adaptable
-

14. Objetivo

Este archivo deja un mapa claro para que puedas desarrollar interfaces que funcionen en cualquier device, sin depender de hacks improvisados ni layout fixes a último minuto.